



Instituto Tecnológico y de Estudios Superiores de Monterrey
Campus Estado de México

M1. Actividad

Juan Pablo Solís Gómez A01800430

Israel González Huerta A01751433

Modelación de sistemas multiagentes con gráficas computacionales (Grupo 302)

Profesor:
Jorge Adolfo Ramírez Uresti

Viernes 14 de noviembre del 2025

Descripción del problema

Se busca desarrollar un sistema Multiagente en Python utilizando la librería Mesa. El objetivo es simular el comportamiento de un robot de limpieza dentro de una habitación de tamaño $M \times N$. El conjunto de agentes (en este caso, los robots) debe limpiar celdas inicialmente sucias, tomando decisiones basadas en las condiciones del medio ambiente.

El espacio o la habitación dividido en celdas, donde:

- Un porcentaje de las celdas está sucio al inicio
- Varios robots empiezan desde la celda (1, 1)
- En cada paso, el robot revisa:
 1. Si la celda está sucia, la limpia
 2. Si la celda está limpia, elige una nueva celda a la cual moverse, si la celda no tiene otro agente, se mueve.

La simulación se detiene cuando:

- Todas las celdas están limpias
- Se excede el tiempo máximo de ejecución (segundos).

Teniendo en cuenta el contexto anterior, correremos nuestra implementación de la simulación variando el número de agentes y el tamaño del espacio para medir la eficiencia según la variación de estas condiciones.

Simulación

Se realizó un programa que acatará a la situación y se crearon simulaciones utilizando un tablero de 20 * 20 celdas, 65% de las celdas sucias y un tiempo máximo de ejecución de 5 segundos.

Simulación con 3 agentes

```
Simulación terminada.  
Pasos ejecutados: 1765  
Celdas sucias restantes: 0  
Porcentaje de celdas limpias: 100.0  
Movimientos totales: 5005  
Tiempo de ejecución: 0.1968segundos
```

Simulación con 10 agentes

```
Simulación terminada.  
Pasos ejecutados: 590  
Celdas sucias restantes: 0  
Porcentaje de celdas limpias: 100.0  
Movimientos totales: 5390  
Tiempo de ejecución: 0.0754segundos
```

Simulación con 20 agentes

```
Simulación terminada.  
Pasos ejecutados: 333  
Celdas sucias restantes: 0  
Porcentaje de celdas limpias: 100.0  
Movimientos totales: 5724  
Tiempo de ejecución: 0.0697segundos
```

Simulación con 100 agentes

Simulación terminada.
Pasos ejecutados: 338
Celdas sucias restantes: 0
Porcentaje de celdas limpias: 100.0
Movimientos totales: 15732
Tiempo de ejecución: 0.1244segundos

En un tablero relativamente pequeño (solo 400 celdas) se ve una conducta interesante para optimización en relación con la cantidad de agentes y tiempo de ejecución. Al trabajar con solo 3 agentes se obtuvo el mayor tiempo de ejecución, esto se puede explicar en la forma en que los agentes dependen de la función pseudoaleatoria para cubrir todas las celdas sucias del tablero. Conforme aumenta la cantidad de agentes a 10 y 20 se puede apreciar que el tiempo disminuye considerablemente, esto se puede explicar en parte por la forma de movimiento del conjunto, a pesar de que no se visualiza, dadas las restricciones en el movimiento de los agentes y la posición inicial, se puede asumir que inicialmente los agentes se mueven como un abanico, dispersándose de [1, 1] y limpiando toda el área. Según disminuye la densidad de agentes en un área, entonces gana mayor peso la probabilidad de que exploren las celdas correctas en el tiempo de ejecución, pero aquí emerge otro patrón.

Con una cantidad suficiente de agentes, estos empiezan a bloquear los movimientos de los otros, sospechamos que esto, en conjunto con los movimientos pseudoaleatorios genera ciertas “áreas limpias” que son ocupadas por agentes acumulados. Al bloquear ciertas áreas y haberlas limpiado, los agentes que lleguen después tenderán a moverse a otros espacios que todavía no hayan sido limpiados por mera probabilidad. Conforme avanza la simulación, los bloques de agentes tenderán a dispersarse y al encontrar otro agente tenderán a separarse, de este modo se puede esperar que los agentes mantengan una distancia promedio entre sí y eventualmente cubran (y por lo tanto limpien) de manera distribuida todo el tablero.

Por su parte, al aumentar drásticamente la cantidad de agentes la cantidad de movimientos totales y tiempo de ejecución aumentan mucho más que con solo 10 y 20 agentes. Esto se puede explicar tomando en cuenta que el abanico inicial de agentes ahora conforma la mayor parte de la simulación. Los primeros agentes en moverse ocupan las celdas cercanas, pero en cada step todos los otros agentes intentan moverse y se encuentran rodeados de agentes, así que no se mueven. En esas condiciones, la tasa total de agentes que se mueven y la cantidad de celdas que revisan no aumenta mucho, pero todos los agentes nuevos representan operaciones computacionales adicionales que finalmente no agregan a la limpieza.

A continuación se consideran un par de simulaciones adicionales para contrastar el rendimiento.

Tablero de 50 * 50 celdas con 3 agentes

Simulación terminada.
Pasos ejecutados: 8133
Celdas sucias restantes: 265
Porcentaje de celdas limpias: 89.4
Movimientos totales: 23036
Tiempo de ejecución: 5.0002segundos

Tablero de 50 * 50 celdas con 100 agentes

Simulación terminada.
Pasos ejecutados: 1282
Celdas sucias restantes: 0
Porcentaje de celdas limpias: 100.0
Movimientos totales: 104141
Tiempo de ejecución: 1.2310segundos

En un tablero de 50 * 50 con 3 y 100 agentes se exagera lo previamente señalado sobre la conducta con pocos agentes. Con solo 3 agentes, en un tablero tan grande y sujetos a la probabilidad, no son capaces de terminar de limpiar el tablero en 5 segundos. Al aumentar la cantidad de agentes, disminuyen la cantidad de pasos y tiempo de ejecución, mientras que se alcanza un 100% de limpieza en los tableros.

Conclusiones

Con estas simulaciones podemos concluir que (dependiendo del problema) existe una cantidad computacionalmente óptima de agentes que tener para resolver un problema. Antes de llegar a esta cantidad, la simulación tardará porque los agentes tardan en cumplir con sus objetivos y si hay demasiados agentes dejan de haber aumentos en eficiencia, pero el consumo de recursos computacionales permanece. Esto se podría mitigar con optimizaciones en el programa, pero aun así debe de ser considerado.